

RAPPORT D'AUDIT DE SECURITE

ONIT-PNG

Observatoire National Intelligent des Telecommunications

Audit complet de securite de la plateforme ONIT-PNG, couvrant l'authentification, le controle d'accès RBAC, les API, la configuration infrastructure et les composants frontend. 27 vulnerabilites identifiees avec plan d'action priorise.

ORGANISATION

ARPT - Autorite de Regulation des Postes et Telecommunications

DATE

17 mai 2026

CLASSIFICATION

Confidentiel

VERSION

1.0

Table des matieres

1. Resume executif	3
2. Presentation de la plateforme ONIT-PNG	3
2.1 Architecture technique	3
2.2 Modele de donnees et roles	4
3. Vulnerabilites critiques	4
3.1 VULN-01 : Secret JWT previsible (NEXTAUTH_SECRET)	4
3.2 VULN-02 : Mot de passe partage pour tous les utilisateurs seed	5
3.3 VULN-03 : Identifiants exposes dans le code client	5
4. Vulnerabilites elevees	6
4.1 VULN-04 : Controle d'accès RLS inefficace pour le scope public_only	6
4.2 VULN-05 : Endpoints de scores et dashboard accessibles sans authentication	7
4.3 VULN-06 : Absence de rate limiting sur l'authentification	7
4.4 VULN-07 : Requete Prisma avec contains vulnérable à l'injection	8
4.5 VULN-08 : Signalement public sans protection anti-abus	8
5. Vulnerabilites moyennes	9
5.1 VULN-09 : SQLite en production	9
5.2 VULN-10 : Absence d'en-tete Content-Security-Policy	9
5.3 VULN-11 : CORS non configure	9
5.4 VULN-12 : Absence de validation du mot de passe actuel lors du changement	10
5.5 VULN-13 : Pas de politique de complexite des mots de passe	10
5.6 VULN-14 : Absence de protection CSRF sur les mutations	10
5.7 VULN-15 : Donnees de test dans le code client (mock-data.ts)	11
5.8 VULN-16 : Logs d'erreur trop verbeux en production	11
5.9 VULN-17 : Reset de mot de passe sans invalidation de session	11
6. Vulnerabilites faibles et points d'attention	12
6.1 VULN-18 : Absence de rotation des sessions JWT	12
6.2 VULN-19 : Absence d'endpoint de sante dedie	12
6.3 VULN-20 : Pas d'expiration des alertes et de nettoyage automatique	12
6.4 VULN-21 : Absence d'audit logging sur les lectures de donnees sensibles	12
6.5 VULN-22 : Validation incomplete du schema de projet	13
6.6 VULN-23 : Pas de 2FA pour les comptes à privileges	13
6.7 VULN-24 : Headers de securite incomplets	13
6.8 VULN-25 : Pas de politique de retention des donnees personnelles	13
6.9 VULN-26 : Absence de tests de securite automatises	14
6.10 VULN-27 : Dependance à aws-sdk v2 deprecated	14

7. Points positifs et bonnes pratiques	14
Validation Zod systematique avec stripHtml	14
Systeme RBAC granulaire	14
Headers de securite de base	14
Audit logging des actions sensibles	15
Docker securise (build multi-stage, utilisateur non-root)	15
Configuration Next.js securisee	15
Protection contre les injections SQL via Prisma	15
Gestion des sessions JWT avec cookies securises	15
8. Plan d'action recommande	15
9. Matrice de vulnerabilites complete	16
10. Conclusion et perspectives	18

1. Resume executif

Ce rapport presente les resultats de l'audit de securite complet de la plateforme ONIT-PNG (Observatoire National Intelligent des Telecommunications), developpee pour l'ARPT (Autorite de Regulation des Postes et Telecommunications) de la Republique de Guinee. L'audit a ete realise sur l'ensemble de la base de code, couvrant les routes API, le systeme d'authentification, le controle d'accès basé sur les rôles (RBAC), la configuration de l'infrastructure, et les composants frontend.

La plateforme ONIT-PNG est une application Next.js 16 utilisant Prisma ORM avec SQLite, NextAuth v4 pour l'authentification, et un systeme RBAC personnalise avec des politiques de controle d'accès aux donnees (Data Access Policies). L'application gere des donnees sensibles de telecommunications incluant des mesures de qualite de service (QoS), des scores d'operateurs, des alertes de zones blanches, et des rapports reglementaires. La nature de ces donnees, qui touche a la regulation nationale et aux informations commerciales des operateurs, impose des exigences de securite elevees.

L'audit a identifie **27 vulnerabilites et points d'amelioration**, dont 3 de severite critique, 5 de severite elevee, 9 de severite moyenne, et 10 de severite faible ou informationnelle. Les problemes les plus preoccupants concernent la gestion des secrets cryptographiques, les faiblesses du systeme de controle d'accès RBAC, l'utilisation de SQLite en production, et l'absence de certaines protections de securite essentielles comme les en-tetes CSP et la limitation du debit sur les endpoints critiques.

Severite	Nombre	Exemples cles
CRITIQUE	3	Secret JWT previsibles, mot de passe partage, credentials exposes cote client
ELEVEE	5	RLS inefficace pour PUBLIC, scores accessibles sans auth, pas de rate limiting auth
MOYENNE	9	SQLite en production, absence CSP, CORS non configure, signalement public sans protection
FAIBLE/INFO	10	Pas de rotation de session, logging insuffisant, absence d'endpoint sante

Tableau 1 : Synthese des vulnerabilites identifiees par severite

2. Presentation de la plateforme ONIT-PNG

2.1 Architecture technique

La plateforme ONIT-PNG est construite sur une architecture monolithique Next.js 16 avec rendu cote serveur (SSR) et routes API integrees. Le framework utilise React 19 pour le frontend, avec une

interface sombre (dark mode) et un systeme de navigation par onglets adaptatif selon le role del'utilisateur. L'application est depoyee via Docker avec un build multi-stage et une base de donneesSQLite embarquee, ce qui simplifie le deploiement mais pose des questions de scalabilite et de securiteen production.

Composant	Technologie	Version
Framework	Next.js	16.1.1
Frontend	React + Recharts + Leaflet	19.0
ORM	Prisma	6.11.1
Base de donnees	SQLite	Embarquee
Authentification	NextAuth v4 (Credentials)	4.x
Validation	Zod	4.4.3
Hash mots de passe	bcryptjs	3.0.3
Conteneurisation	Docker (multi-stage)	Node 20 Alpine

Tableau 2 : Stack technique de la plateforme

2.2 Modele de donnees et roles

Le schema Prisma definit 9 roles distincts allant de SUPER_ADMIN a PUBLIC, avec un systeme de permissions granulaires base sur des tuples (ressource, action). Les politiques de controle d'accès aux donnees (DataAccessPolicy) implementent un filtrage de type Row-Level Security (RLS) avec des scopes : all, public_only, own_region, own_org. Le modele comprend 13 entites principales couvrant les utilisateurs, les roles, les operateurs telecoms, les regions, les campagnes de mesures, les mesures QoS, les scores d'opérateurs, les alertes, les rapports, et les journaux d'audit. Chaque role possede des permissions specifiques qui determinent l'accès aux differentes fonctionnalites et aux donnees selon le principe du moindre privilege.

3. Vulnerabilites critiques

3.1 VULN-01 : Secret JWT previsible (NEXTAUTH_SECRET)

Severite : CRITIQUE | **Fichier** : .env | **CWE** : CWE-321 (Use of Hard-coded Cryptographic Key)

Le fichier .env contient un secret JWT previsible :
onit-png-secret-key-CHANGE-ME-IN-PRODUCTION. Ce secret est utilise pour signer et verifier tous les tokens JWT de session. Un attaquant connaissant cette valeur peut forger des tokens d'authentification valides pour n'importe quel utilisateur, y compris SUPER_ADMIN, et obtenir un

accès complet à la plateforme. Ce type d'attaque est particulièrement dangereux car il ne laisse aucune trace dans les journaux d'audit, le token étant cryptographiquement valide.

Bien que le fichier `docker-compose.yml` exige que `NEXTAUTH_SECRET` soit défini via `${NEXTAUTH_SECRET: ?}`, le fichier `.env` actif contient toujours la valeur par défaut. De plus, le fichier `.env.example` contient la valeur `changez-cette-valeur-en-production`, ce qui suggère que cette valeur n'a jamais été changée dans les environnements de déploiement.

Recommandation :

Générer un secret aléatoire fort pour chaque environnement avec `openssl rand -base64 32` et le stocker dans un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager, ou variables d'environnement sécurisées). Ne jamais committer de secrets dans le dépôt de code. Ajouter `.env` au fichier `.gitignore` et implémenter une vérification au démarrage qui refuse de lancer l'application si le secret correspond à une valeur par défaut connue.

3.2 VULN-02 : Mot de passe partagé pour tous les utilisateurs seed

Severite : CRITIQUE | **Fichier :** `prisma/seed.ts` | **CWE :** CWE-259 (Use of Hard-coded Password)

Le script de seeding (`prisma/seed.ts`) crée 10 utilisateurs avec le même mot de passe `Admin@2026!`, qui est en outre imprimé dans la console lors de l'exécution. Parmi ces utilisateurs, on trouve des comptes critiques comme `admin@arpt.gn` (SUPER_ADMIN), `dg@arpt.gn` (Directeur General), et des comptes opérateurs comme `tech@orange.gn`. Si l'un de ces comptes est compromis, l'attaquant a accès à tous les autres comptes, créant un effet domino particulièrement dangereux pour les comptes à privilèges élevés.

De plus, le fait que le mot de passe soit affiché en clair dans les logs du seed crée un risque supplémentaire d'exposition dans les systèmes de logging centralisés, les artefacts CI/CD, et les captures d'écran de débogage. Ce motif est contraire aux bonnes pratiques de sécurité qui exigent des mots de passe uniques et générés aléatoirement pour chaque compte.

Recommandation :

Générer des mots de passe uniques et aléatoires pour chaque utilisateur lors du seed, et les stocker de manière sécurisée (vault, fichier chiffré). Ne jamais afficher les mots de passe dans les logs. Pour les environnements de démonstration, utiliser des identifiants clairement marqués comme non-productifs avec des mots de passe temporaires qui expirent. Implémenter une politique de rotation des mots de passe obligeant les utilisateurs à changer leur mot de passe à la première connexion.

3.3 VULN-03 : Identifiants exposés dans le code client

Severite : CRITIQUE | **Fichier** : src/components/login-modal.tsx | **CWE** : CWE-200 (Exposure of Sensitive Information)

Le composant login-modal.tsx contient en dur la liste des 10 adresses email des comptes de demonstration avec leurs roles associes, cote client (navigateur). N'importe quel utilisateur consultant le code source de la page peut voir ces emails internes, ce qui revele la structure organisationnelle de l'ARPT et les roles de chaque utilisateur. Combine avec VULN-02 (mot de passe partage), cela permet a un attaquant de se connecter avec n'importe quel compte sans effort de devinement.

L'exposition des emails internes facilite egalement les attaques d'ingenierie sociale cibles (spear phishing), car l'attaquant connait les adresses email et les fonctions de chaque personnel. Dans un contexte regulatorire ou les operateurs telecoms sont egalement surveilles par la plateforme, cette fuite d'information est particulierement problematique.

Recommandation :

Supprimer la liste de comptes de demonstration du code client. Si une fonctionnalite de connexion rapide est necessaire pour les demonstrations, l'implementer via un endpoint API securise qui ne retourne que les comptes autorises en environnement de developpement, sans exposer les adresses email reelles. En production, cette fonctionnalite doit etre completement desactivee.

4. Vulnerabilites elevees

4.1 VULN-04 : Controle d'accès RLS inefficace pour le scope public_only

Severite : ELEVEE | **Fichier** : src/lib/rbac.ts | **CWE** : CWE-863 (Incorrect Authorization)

Les fonctions `getAccessibleRegions()` et `getAccessibleOperators()` dans `rbac.ts` retournent la liste complete de toutes les regions et operateurs lorsque le scope est `public_only`. Cela signifie qu'un utilisateur non authentifie avec le role PUBLIC peut acceder aux memes donnees filtrees qu'un utilisateur authentifie, contournant completement la politique de restriction d'accès. Le scope `public_only` devrait ne retourner que les donnees explicitement marquees comme publiques (`isPublic=true`), et non l'ensemble des identifiants de regions et operateurs.

Cette vulnerabilite a un impact direct sur la confidentialite des donnees operateurs, car les endpoints `/api/dashboard`, `/api/scoring`, `/api/scores` et `/api/map` retournent des donnees detaillees (scores, mesures QoS, zones blanches) a tout utilisateur anonyme.

Recommandation :

Modifier le comportement du scope `public_only` pour ne retourner que les donnees explicitement marquees comme publiques. Implementer un filtrage au niveau des requetes Prisma en ajoutant des

conditions `where: { isPublic: true }` pour les rapports, et des agregations anonymisees pour les scores et mesures. Les regions et operateurs ne doivent etre accessibles que par leurs codes/noms, jamais par leurs identifiants internes.

4.2 VULN-05 : Endpoints de scores et dashboard accessibles sans authentication

Severite : ELEVEE | **Fichiers :** `src/app/api/scores/route.ts`, `src/app/api/dashboard/route.ts` | **CWE :** CWE-306 (Missing Authentication)

Les endpoints `/api/scores` et `/api/dashboard` acceptent des requetes sans authentication. Lorsqu'aucune session n'est trouvee, le role est defini par default a `PUBLIC`, mais le code continue a executer les requetes de base de donnees avec le scope `public_only` qui, comme decrit dans VULN-04, retourne toutes les donnees. Un attaquant anonyme peut donc extraire l'integralite des scores d'operateurs, des KPIs nationaux, et des statistiques regionales, ce qui constitue une fuite massive de donnees sensibles dans un contexte regulatorie.

Recommandation :

Exiger l'authentification pour tous les endpoints sauf ceux explicitement conqus pour l'acces public. Pour les donnees destinees au public, creer des endpoints dedies qui ne retournent que des agregations et des donnees anonymisees, sans identifiants internes ni detail operateur. Ajouter une verification de session dans le middleware pour les routes API sensibles.

4.3 VULN-06 : Absence de rate limiting sur l'authentification

Severite : ELEVEE | **Fichier :** `src/middleware.ts` | **CWE :** CWE-307 (Improper Restriction of Excessive Authentication Attempts)

Bien que le middleware implemente un rate limiting pour les tentatives de connexion (5 tentatives / 15 minutes par IP), ce mecanisme utilise un stockage en memoire (Map JavaScript) qui est perdu a chaque redemarrage du serveur et ne fonctionne pas avec plusieurs instances de l'application. En deploiement Docker avec replication ou derriere un load balancer, un attaquant peut contourner cette protection simplement en repartissant ses tentatives sur differentes instances ou en utilisant differentes adresses IP (attaque distribuee).

De plus, le rate limiting ne s'applique qu'au endpoint `/api/auth/callback/credentials`, laissant les autres endpoints d'authentification comme `/api/auth/register` et `/api/auth/forgot-password` sans protection contre les abus.

Recommandation :

Implementer un rate limiting persistant utilisant Redis ou la base de donnees comme backing store. Configurer egalement un rate limiting au niveau du reverse proxy (Nginx) ou du load balancer pour une

protection independante du nombre d'instances. Etendre le rate limiting a tous les endpoints d'authentification et considerer l'implementation d'un mecanisme de verrouillage de compte apres un nombre defini d'echecs.

4.4 VULN-07 : Requete Prisma avec contains vulnerable a l'injection

Severite : ELEVEE | **Fichier** : src/lib/rbac.ts | **CWE** : CWE-89 (SQL Injection)

Dans la fonction `getAccessibleOperators()`, la ligne `where: { nom: { contains: organization.split(' ')[0] } }` passe une valeur derivee de l'input utilisateur directement a un filtre Prisma `contains`. Bien que Prisma utilise des requetes parametrees qui previennent l'injection SQL classique, l'utilisation de `contains` avec des metacaracteres SQLite (`%`, `_`) peut permettre des abus de type LIKE injection. Un attaquant pourrait manipuler le champ `organization` de son profil pour matcher des operateurs auxquels il ne devrait pas avoir acces.

Recommandation :

Remplacer le filtrage par `contains` par une correspondance exacte ou utiliser une table de mapping explicite entre organisations et operateurs. Si le `contains` est necessaire, echapper les metacaracteres SQLite dans l'input avant de le passer a Prisma.

4.5 VULN-08 : Signalement public sans protection anti-abus

Severite : ELEVEE | **Fichier** : src/app/api/alerts/route.ts | **CWE** : CWE-799 (Improper Control of Interaction Frequency)

L'endpoint `POST /api/alerts` permet la creation d'alertes de type `SIGNALEMENT_PUBLIC` sans authentification. Bien que cette fonctionnalite soit intentionnelle (permettre aux citoyens de signaler des problemes de couverture), l'absence de toute protection anti-abus (rate limiting, CAPTCHA, validation d'email) permet a un attaquant de spammer le systeme avec des faux signalements, inondant les equipes de supervision et potentiellement masquant de vraies alertes critiques.

Recommandation :

Implementer un rate limiting strict sur les signalements publics (par exemple 3 signalements par heure par adresse IP), ajouter un CAPTCHA pour les soumissions non authentifiees, et considerer l'implementation d'un systeme de verification par email ou SMS pour valider les signalements. Mettre en place un systeme de deduplication pour eviter les signalements multiples du meme probleme.

5. Vulnerabilites moyennes

5.1 VULN-09 : SQLite en production

Severite : MOYENNE | **CWE** : Voir detail

La base de donnees SQLite est embarquee dans le conteneur Docker avec un fichier local (file:/app/db/onit-png.db). SQLite ne supporte pas le acces concurrents en ecriture, les connexions reseau, le chiffrement au repos, ou les sauvegardes a chaud. En cas de panne du conteneur, les donnees peuvent etre corrompues si une ecriture etait en cours. Pour une plateforme regulatorire nationale manipulant des donnees sensibles de telecommunications, PostgreSQL ou MySQL sont indispensables.

Recommandation :

Migrer vers PostgreSQL avec connexion chiffree (SSL/TLS), chiffrement au repos, et sauvegardes automatiques. Configurer Prisma avec le provider postgresql et utiliser un service managed de base de donnees en production.

5.2 VULN-10 : Absence d'en-tete Content-Security-Policy

Severite : MOYENNE | **CWE** : Voir detail

Le middleware ne configure pas d'en-tete Content-Security-Policy (CSP), laissant la page vulnérable aux attaques XSS et aux injections de scripts malveillants. Sans CSP, n'importe quel script inline ou externe peut s'exécuter dans le contexte de la page, permettant le vol de sessions et de données sensibles.

Recommandation :

Ajouter un en-tete CSP restrictif dans le middleware qui limite les sources de scripts, styles, et connexions aux domaines approuves. Utiliser une politique nonce-based pour les scripts inline necessaires.

5.3 VULN-11 : CORS non configure

Severite : MOYENNE | **CWE** : Voir detail

Aucune configuration CORS n'est presente sur les routes API. Par default, les navigateurs appliquent la same-origin policy, mais l'absence de configuration explicite peut permettre des requetes cross-origin non autorisees dans certains scenarios. Si l'application est hebergee derriere un proxy ou un CDN, des headers CORS permissifs par default pourraient exposer les API a des tiers.

Recommandation :

Configurer explicitement les headers CORS pour n'autoriser que les origines approuvees. Restreindre les methodes HTTP autorisees et les headers de requete.

5.4 VULN-12 : Absence de validation du mot de passe actuel lors du changement

Severite : MOYENNE | **CWE** : Voir detail

L'endpoint PUT /api/users/[id] permet de modifier le mot de passe d'un utilisateur sans exiger la validation du mot de passe actuel. Bien que l'acces soit limite aux admins ou a l'utilisateur lui-meme, un attaquant ayant acces a une session compromise peut changer le mot de passe sans connaitre l'ancien, verrouillant le proprietaire legitime hors de son compte.

Recommandation :

Exiger la validation du mot de passe actuel pour tout changement de mot de passe par l'utilisateur lui-meme. Pour les admins, implementer une verification supplementaire (2FA, confirmation par email).

5.5 VULN-13 : Pas de politique de complexite des mots de passe

Severite : MOYENNE | **CWE** : Voir detail

Le schema de validation des mots de passe (validators.ts) exige uniquement un minimum de 8 caracteres, sans exigences de complexite (majuscules, minuscules, chiffres, caracteres speciaux). Cela permet des mots de passe faibles comme 'password1' qui sont vulnerables aux attaques par dictionnaire et par force brute.

Recommandation :

Renforcer le schema Zod avec des exigences de complexite : au moins une majuscule, une minuscule, un chiffre et un caractere special. Verifier les mots de passe contre les listes de mots de passe compromis (Have I Been Pwned API) et imposer une longueur minimale de 12 caracteres pour les comptes a privileges.

5.6 VULN-14 : Absence de protection CSRF sur les mutations

Severite : MOYENNE | **CWE** : Voir detail

Les routes API POST/PUT/PATCH/DELETE n'ont pas de protection CSRF explicite au-dela du token CSRF de NextAuth. Les cookies SameSite=Lax offrent une protection partielle mais pas complete, particulierement pour les sous-domaines et les attaques via des methodes HTTP inhabituelles.

Recommandation :

Implementer des tokens anti-CSRF pour toutes les mutations, ou utiliser le mecanisme CSRF de NextAuth de maniere coherente. Considerer l'utilisation de cookies SameSite=Strict pour les endpoints

sensibles.

5.7 VULN-15 : Donnees de test dans le code client (mock-data.ts)

Severite : MOYENNE | **CWE** : Voir detail

Le fichier `src/lib/mock-data.ts` existe dans la base de code, suggérant la presence de donnees de test dans le build de production. Si ces donnees sont incluses dans le bundle client, elles peuvent reveler la structure des donnees et des informations internes.

Recommandation :

Exclure les fichiers de donnees de test du build de production via des imports conditionnels ou des variables d'environnement. Verifier que le bundle final ne contient aucune donnee de test.

5.8 VULN-16 : Logs d'erreur trop verbeux en production

Severite : MOYENNE | **CWE** : Voir detail

Plusieurs routes API utilisent `console.error()` avec l'objet erreur complet, ce qui peut exposer des informations sensibles (stack traces, requetes SQL, variables d'environnement) dans les logs de production. Le `logger.ts` enregistre egalement des metadonnees potentiellement sensibles dans les logs.

Recommandation :

Implementer un logging structure qui masque les informations sensibles en production. Ne logger que le type d'erreur et un identifiant de trace, sans stack trace ni details internes. Utiliser un niveau de log `WARN` pour les erreurs utilisateur et `ERROR` uniquement pour les erreurs systeme.

5.9 VULN-17 : Reset de mot de passe sans invalidation de session

Severite : MOYENNE | **CWE** : Voir detail

Le endpoint `/api/auth/reset-password` ne supprime pas les sessions existantes apres un changement de mot de passe. Un attaquant ayant compromis un compte peut continuer a acceder au compte meme apres que le proprietaire legitime a change son mot de passe.

Recommandation :

Invalider toutes les sessions existantes lors d'un changement de mot de passe. Implementer un mecanisme de revocation de session cote serveur en utilisant la table `Session` du schema `Prisma`.

6. Vulnerabilites faibles et points d'attention

6.1 VULN-18 : Absence de rotation des sessions JWT

Les sessions JWT (strategie JWT, maxAge 8h) ne peuvent pas etre revoquees cote serveur. Un utilisateur dont le token est vole peut l'utiliser pendant toute la duree de validite malgre une deconnexion ou un changement de mot de passe.

Recommandation :

Implementer un mecanisme de liste noire de tokens ou un systeme de version de session dans la base de donnees pour permettre la revocation immediate.

6.2 VULN-19 : Absence d'endpoint de sante dedie

Il n'existe pas d'endpoint /api/health dedie. Le healthcheck Docker utilise /api/auth/session qui ne verifie pas l'etat de la base de donnees ou des services externes. Un probleme de base de donnees ne sera pas detecte par le healthcheck.

Recommandation :

Creer un endpoint /api/health qui verifie la connexion base de donnees, l'etat de la session, et les eventuels services externes.

6.3 VULN-20 : Pas d'expiration des alertes et de nettoyage automatique

Les alertes n'ont pas de date d'expiration et le systeme ne nettoie pas automatiquement les vieilles alertes resolues. Avec le temps, la base de donnees SQLite va grossir indefiniment, degradant les performances.

Recommandation :

Implementer un nettoyage automatique des alertes resolues de plus de X jours et des journaux d'audit anciens. Ajouter un index sur les champs de date pour les requetes frequentes.

6.4 VULN-21 : Absence d'audit logging sur les lectures de donnees sensibles

L'audit logging est implemente pour les ecritures (CREATE, UPDATE, DELETE) mais pas pour les lectures de donnees sensibles. Un utilisateur malveillant peut extraire massivement des donnees sans laisser de trace dans les journaux d'audit.

Recommandation :

Etendre l'audit logging aux lectures de donnees sensibles (scores, mesures, rapports confidentiels).
Implementer des alertes sur les volumes de lecture anormaux.

6.5 VULN-22 : Validation incomplete du schema de projet

Le schema projectSchema dans validators.ts contient une typo : 'CNCELLED' au lieu de 'CANCELLED' dans l'enum status. Cela empeche la creation de projets avec le statut CANCELLED et peut causer des erreurs silencieuses.

Recommandation :

Corriger la typo et ajouter des tests unitaires pour les schemas de validation.

6.6 VULN-23 : Pas de 2FA pour les comptes a privileges

Les comptes SUPER_ADMIN, DG et DGA n'ont pas d'authentification multi-facteurs (2FA). Ces comptes ont un acces complet a toutes les donnees et fonctionnalites, et leur compromission aurait un impact catastrophique.

Recommandation :

Implementer la 2FA (TOTP ou WebAuthn) pour tous les comptes a privileges eleves. Rendre la 2FA obligatoire pour les roles SUPER_ADMIN, DG, DGA et DIRECTEUR_TECHNIQUE.

6.7 VULN-24 : Headers de securite incomplets

Les headers X-Frame-Options, X-Content-Type-Options et Referrer-Policy sont bien configures, mais il manque Strict-Transport-Security (HSTS), qui est essentiel pour forcer les connexions HTTPS en production.

Recommandation :

Ajouter l'en-tete Strict-Transport-Security avec une duree d'au moins 1 an et l'option includeSubDomains. Considerer l'ajout de preload pour une protection maximale.

6.8 VULN-25 : Pas de politique de retention des donnees personnelles

Les utilisateurs supprimes (soft delete via deletedAt dans l'ancien schema, ou desactivation isActive dans le schema actuel) conservent leurs donnees personnelles indefiniment dans la base de donnees, sans politique de retention ou d'anonymisation conforme au RGPD.

Recommandation :

Implementer une politique de retention et d'anonymisation des donnees personnelles conforme au RGPD et a la legislation guineenne sur la protection des donnees.

6.9 VULN-26 : Absence de tests de securite automatisees

Aucun test de securite automatis (SAST, DAST, test d'intrusion) n'est configure dans le pipeline CI/CD. Le fichier package.json ne contient pas de scripts de test.

Recommandation :

Integrer des outils SAST (SonarQube, Semgrep), DAST (OWASP ZAP), et des tests de securite automatisees dans le pipeline CI/CD. Ajouter des tests unitaires et d'integration pour les mecanismes d'authentification et d'autorisation.

6.10 VULN-27 : Dependance a aws-sdk v2 deprecated

L'ancien code utilise le package aws-sdk v2 qui est deprecated. Bien que le code actuel ne semble plus utiliser S3 directement, la presence de cette dependance dans le package.json cree des vulnerabilites potentielles et augmente la surface d'attaque.

Recommandation :

Supprimer les dependances non utilisees du package.json. Si S3 est necessaire, migrer vers @aws-sdk/client-s3 v3 qui est le SDK actif.

7. Points positifs et bonnes pratiques

Malgre les vulnerabilites identifiees, la plateforme ONIT-PNG implemente plusieurs bonnes pratiques de securite qui meritent d'etre soulignees. Ces elements constituent une base solide sur laquelle construire les ameliorations recommandees.

Validation Zod systematique avec stripHtml

Toutes les entrees utilisateur sont validees avec Zod et les tags HTML sont systematiquement supprimees via la fonction stripHtml, prevenant efficacement les attaques XSS par injection de balises. Cette approche coherente a travers tous les endpoints est une excellente pratique.

Systeme RBAC granulaire

L'implementation d'un systeme de controle d'accès basé sur les rôles avec des permissions par ressource et action, associé à des politiques d'accès aux données (RLS), est une architecture solide qui permet un contrôle fin des accès. Le modèle à 9 rôles couvre bien les différents profils d'utilisation de la plateforme.

Headers de securite de base

Le middleware configure les en-tetes X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection, Referrer-Policy: strict-origin-when-cross-origin, et Permissions-Policy, offrant une

protection de base contre le clickjacking, le MIME sniffing, et les fuites de referer.

Audit logging des actions sensibles

Les actions d'écriture (CREATE, UPDATE, DELETE) sont enregistrées dans la table AuditLog avec l'utilisateur, l'action, la ressource, et les détails. Ce journal d'audit est accessible via un endpoint dédié pour les rôles autorisés, permettant la traçabilité et le forensic en cas d'incident.

Docker sécurise (build multi-stage, utilisateur non-root)

Le Dockerfile utilise un build multi-stage avec une image finale Alpine, un utilisateur non-root (nextjs), et un healthcheck. Le docker-compose.yml exige la variable NEXTAUTH_SECRET via \${NEXTAUTH_SECRET:?}, empêchant le démarrage sans secret.

Configuration Next.js sécurisée

Le fichier next.config.ts désactive ignoreBuildErrors, active reactStrictMode, supprime l'en-tête X-Powered-By, et utilise le mode standalone pour la production. Ce sont des configurations recommandées par la documentation de sécurité de Next.js.

Protection contre les injections SQL via Prisma

L'utilisation de Prisma ORM avec des requêtes paramétrées élimine le risque d'injection SQL classique. L'ensemble des requêtes utilise les méthodes de l'ORM plutôt que du SQL brut, ce qui est une bonne pratique fondamentale.

Gestion des sessions JWT avec cookies sécurisés

Les cookies de session sont configurés avec httpOnly, sameSite=lax, et secure en production HTTPS. Le nom du cookie change entre __Secure- et le préfixe normal selon l'environnement, et la durée de session est limitée à 8 heures.

8. Plan d'action recommandé

Le plan d'action suivant priorise les corrections en fonction de la sévérité des vulnérabilités et de l'impact potentiel sur la sécurité de la plateforme. Les délais estimés supposent une équipe de développement disponible et compétente.

Priorité	Actions	Vulnérabilités	Délai estimé
P0 - Urgent	Changer NEXTAUTH_SECRET, supprimer les credentials du code client, générer des mots de passe uniques	VULN-01, 02, 03	24-48h

P1 - Elevee	Corriger le RLS public_only, exiger l'auth sur les endpoints sensibles, rate limiting persistant, corriger l'injection contains, proteger les signalements publics	VULN-04, 05, 06, 07, 08	1-2 semaines
P2 - Moyenne	Migrer vers PostgreSQL, ajouter CSP/CORS, renforcer la validation, audit logging des lectures, corriger les typos, nettoyer les deps	VULN-09 a 17	2-4 semaines
P3 - Faible	Ajouter 2FA, HSTS, rotation de sessions, tests de securite, politique de retention, endpoint sante, nettoyage auto	VULN-18 a 27	1-2 mois

Tableau 3 : Plan d'action priorise

9. Matrice de vulnerabilites complete

Le tableau ci-dessous resume l'ensemble des vulnerabilites identifiees lors de cet audit, avec leur identification, leur severite, le composant affecte, et la categorie CWE associee. Cette matrice permet une vue synthetique de l'ensemble des problemes de securite et facilite le suivi des corrections.

ID	Severite	Titre	Composant	CWE
VULN-01	CRITIQUE	Secret JWT previsible	.env	CWE-321
VULN-02	CRITIQUE	Mot de passe partage (seed)	prisma/seed.ts	CWE-259
VULN-03	CRITIQUE	Credentials exposes client	login-modal.tsx	CWE-200
VULN-04	ELEVEE	RLS inefficace public_only	src/lib/rbac.ts	CWE-863
VULN-05	ELEVEE	Endpoints sans authentication	API routes	CWE-306
VULN-06	ELEVEE	Rate limiting en memoire	middleware.ts	CWE-307
VULN-07	ELEVEE	Injection via contains	src/lib/rbac.ts	CWE-89
VULN-08	ELEVEE	Signalement sans anti-abus	alerts/route.ts	CWE-799
VULN-09	MOYENNE	SQLite en production	schema.prisma	CWE-311

ID	Severite	Titre	Composant	CWE
VULN-10	MOYENNE	Absence CSP	middleware.ts	CWE-693
VULN-11	MOYENNE	CORS non configure	API routes	CWE-942
VULN-12	MOYENNE	Pas de validation mot de passe actuel	users/route.ts	CWE-620
VULN-13	MOYENNE	Politique mots de passe faible	validators.ts	CWE-521
VULN-14	MOYENNE	Pas de protection CSRF	API routes	CWE-352
VULN-15	MOYENNE	Donnees test en production	mock-data.ts	CWE-489
VULN-16	MOYENNE	Logs trop verbeux	API routes	CWE-532
VULN-17	MOYENNE	Pas d'invalidation session	auth routes	CWE-613
VULN-18	FAIBLE	Pas de rotation sessions JWT	auth config	CWE-613
VULN-19	FAIBLE	Pas d'endpoint sante	API routes	CWE-754
VULN-20	FAIBLE	Pas de nettoyage alertes	data model	CWE-400
VULN-21	FAIBLE	Pas d'audit logging lectures	rbac.ts	CWE-778
VULN-22	FAIBLE	Typo dans validateur	validators.ts	CWE-20
VULN-23	FAIBLE	Pas de 2FA pour admins	auth system	CWE-308
VULN-24	FAIBLE	HSTS absent	middleware.ts	CWE-319
VULN-25	FAIBLE	Pas de retention donnees	data model	CWE-312
VULN-26	FAIBLE	Pas de tests securite	CI/CD	CWE-1357
VULN-27	FAIBLE	Dependance aws-sdk v2	package.json	CWE-1104

Tableau 4 : Matrice complete des vulnerabilites identifiees

10. Conclusion et perspectives

La plateforme ONIT-PNG presente une architecture de securite globalement coherente avec des fondements solides, notamment le systeme RBAC granulaire, la validation systematique des entrees via Zod avec stripHtml, les headers de securite de base, et l'audit logging des ecritures. Ces elements temoignent d'une reflexion serieuse sur la securite des le stade de la conception.

Cependant, l'audit revele plusieurs vulnerabilites critiques qui necessitent une action immediate, en particulier la gestion des secrets cryptographiques, l'exposition de credentials dans le code client, et les lacunes du systeme de controle d'accès pour les utilisateurs non authentifies. Ces problemes, s'ils sont exploites, pourraient compromettre l'integrite de la plateforme et la confidentialite des donnees sensibles de telecommunications qu'elle gere pour le compte de l'ARPT.

La migration vers PostgreSQL est un prealable indispensable pour un deploiement en production, tant pour des raisons de performance et de fiabilite que de securite (chiffrement au repos, connexions securisees, sauvegardes). L'ajout d'un CSP, du HSTS, et de mecanismes de protection anti-abus completera la posture de securite de l'application.

En suivant le plan d'action recommande, en commençant par les corrections P0 (24-48h) et P1 (1-2 semaines), la plateforme peut atteindre un niveau de securite acceptable pour un deploiement en production. Un audit de penetration externe est recommande apres la mise en oeuvre des corrections pour valider l'efficacite des mesures implementees.